

Predictability is Trainable: Optimizing MoE LLMs to be Forecastable for Disk-Resident Inference

Spencer Churchill
Independent
spence@duck.com

Abstract

Inference engines such as Colibri run 744B-parameter MoE models on consumer hardware by keeping the dense backbone resident and streaming expert weights from disk on demand—a reactive design whose throughput is bounded by fetch latency. Post-hoc predictors forecast expert activations from earlier hidden states, but they chase a model that was never asked to be predictable. We ask whether predictability is trainable: we fold the accuracy of a small expert-activation predictor into the MoE’s own training loss, so the model is optimized jointly for output quality and for being forecastable. At 341M-parameter scale we find: (i) predictability is a trainable, transferable backbone property—a fresh post-hoc predictor trained on the frozen jointly-trained backbone recovers the full gain (+6–7 points top- k hit rate over a linear control, +2.9–5.2 points over a ranking-aware control; both 3 seeds); (ii) the gain is structural, not sharpening—entropy penalties at any strength cannot reach it at matched entropy; (iii) it grows with training while its quality cost vanishes; and (iv) it converts to systems value (+3.2% tok/s and −31% misprefetch waste in a disk-queue-accurate simulator; +5.0% mean tok/s in a purpose-built real engine). Finally, we map the boundary: a LoRA fine-tune on pretrained OLMoE-1B-7B transfers only a small fraction of the effect even at $10\times$ loss pressure—the method is pretraining-time only. **Code:** <https://github.com/splch/predictability-aware-training>

1 Introduction

Disk-resident inference engines have made mixture-of-experts (MoE) models runnable far outside the datacenter. Engines such as Colibri [10] run 744B-parameter MoE models on consumer hardware: the dense backbone stays resident while tens of thousands of routed experts are streamed from disk on demand. The design is fundamentally reactive—an expert fetch begins only after the router reveals that it is needed—so decode throughput is bounded by fetch latency rather than compute. Colibri’s own mitigation is a router-lookahead thread (PILOT) that prefetches next-layer experts, exploiting the fact that routing is roughly 72% predictable one layer ahead on a production model.

A growing line of post-hoc work pushes this further, training predictors that forecast expert activations from earlier hidden states at 93–97% accuracy. But every such system treats the model as fixed: the predictor chases the model. We invert the relationship. The accuracy of an *independent* expert-activation predictor becomes an explicit term in the MoE’s own training loss, regularizing the backbone toward being forecastable at multi-layer lookahead horizons while the predictor simultaneously adapts to the model.¹

Concretely, we train a 341M-parameter GPT-style MoE (16 experts, top-2) with a joint loss $\mathcal{L} = \mathcal{L}_{\text{LM}} + 0.01 \mathcal{L}_{\text{balance}} + \lambda \mathcal{L}_{\text{pred}}$, where $\mathcal{L}_{\text{pred}}$ is a soft cross-entropy between predictor logits and the realized top- k routing set. Gradients from $\mathcal{L}_{\text{pred}}$ reach the backbone only through hidden representations, so the loss shapes representations rather than router weights directly. We then ask four questions: is the resulting predictability a property of the backbone itself (Section 4.2); is it merely router sharpening (Section 4.3); does it survive and grow with training (Section 4.4); and does it convert to systems value (Section 5)? The answers are yes, no, yes, and yes—with a precisely mapped boundary: the effect must be baked in during pretraining (Section 6).

Contributions. We make the following contributions:

- **Predictability is a trainable, transferable backbone property.** Fresh post-hoc predictors trained on the frozen jointly-trained backbone recover the entire gain, so a deployed engine needs no co-training (Table 1).

¹Code, data pipeline, run scripts, and the full audit trail: <https://github.com/splch/predictability-aware-training>

- **The gain is structural, not sharpening.** Entropy-penalty controls at any strength cannot reach the joint model’s accuracy at matched entropy; roughly +2–2.4 points of the advantage is pure structure (Figure 1).
- **The effect is robust and grows with training.** It survives init and data-order variance (± 0.3 pts) and *increases* at $4\times$ tokens, where the quality cost flips to a small gain (Table 2).
- **Predictability converts to systems value, with a mapped boundary.** +3.2% tok/s and -31% misprefetch waste over the post-hoc control at Colibri-like geometry (+1.6%/–27% at ranking-control accuracies), gated by fetch time $\leq$ compute window; +5.0% mean tok/s in a real O_DIRECT engine. A LoRA fine-tune on a pretrained model recovers only a fraction even at $10\times$ loss pressure—the method is pretraining-time only (Table 4).

2 Related Work

Post-hoc expert prediction. A large body of work forecasts expert activations on a frozen model. Pre-attention linear predictors with ranking-aware losses reach 93–97.6% top- k accuracy [22]; cross-layer gate inputs enable edge offloading with 99% cache hits (Fate [5]); trace-trained sequence predictors raise cache hit rates from 17% to 72% at a 10% budget (MoE-Beyond [7]); PreScope [19] adds globally scheduled prefetching. Adjacent offloading and caching systems include Apple’s LLM-in-a-Flash [1], PowerInfer-2 [18], and MoE-Infinity [17]. All of these train predictors *against* a fixed model; none change the model.

Training-time routing modification. StickyMoE [11] adds a differentiable routing-consistency loss for temporal reuse—the closest philosophy to ours, but it optimizes stickiness (token t vs. $t-1$), not forecastability from earlier layers; in our reimplementation it does not buy lookahead predictability (Section 4.6). Oracle-MoE [21] redesigns the routing input for emergent locality without an explicit objective. ReMoE [23] fine-tunes only the router post-hoc. Pre-gated MoE [9] is the closest co-design: a pre-gate module that *replaces* next-layer routing, co-trained end-to-end—but it has no predictor-accuracy objective and no realized router to evaluate forecasts against. Halfway Speculative Decoding [14] applies the same co-design pattern—train the target to be draftable—in token space. Routing regularization for quality or stability [4, 8, 12] never targets inference-time predictability.

Novelty. Prior systems work trains predictors post-hoc against frozen models; prior training-time work optimizes routing temporal locality; Pre-gated MoE co-trains a lookahead gate that replaces routing. To our knowledge, no prior work (a) makes an independent predictor’s accuracy an explicit training objective of the MoE itself, and (b) demonstrates that the resulting predictability is a recoverable property of the backbone’s representations. Where our ranking-aware control reimplements the protocol of Zhu et al. [22] we say so; all comparisons use our implementation, not their published numbers.

3 Method

3.1 Model and training

We train a GPT-style decoder-only MoE: $d=512$, 12 layers, 16 experts per layer with top-2 routing, 341M total / 99M active parameters. Training uses FineWeb-edu [15] tokenized with the GPT-2 BPE [16] as a document-shuffled stream with EOS separators, context length 512, bf16 autocast, for 25M tokens (100M in the scaling run).

3.2 Expert-activation predictors

Predictors are per-(layer, horizon) heads mapping the pre-MoE hidden state at layer l to the router decision at layer $l+h$, for $h \in \{1, 2, 4\}$. We study two families: a *linear* predictor (a single projection) and a *ranking-MLP* (a 2-layer MLP trained with a margin loss requiring the weakest true top- k logit to exceed the strongest intruder by a margin), the latter reimplementing the protocol of Zhu et al. [22].

Table 1: **Main result.** Hit@ k of a fresh linear post-hoc predictor on frozen backbones (chance = 0.125), with validation LM loss; mean (std) over 3 seeds. Joint training improves predictability at every horizon for a small quality cost.

Backbone	$h=1$ ($\uparrow$)	$h=2$ ($\uparrow$)	$h=4$ ($\uparrow$)	Val LM ($\downarrow$)
Baseline ($\lambda=0$)	0.826 (0.003)	0.797 (0.002)	0.732 (0.004)	5.767 (0.006)
Joint ($\lambda=0.3$)	0.888 (0.002)	0.865 (0.002)	0.796 (0.004)	5.790 (0.006)

3.3 Joint training objective

The training loss combines three terms:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + 0.01 \mathcal{L}_{\text{balance}} + \lambda \mathcal{L}_{\text{pred}} \quad (1)$$

where $\mathcal{L}_{\text{LM}}$ is the language-modeling loss, $\mathcal{L}_{\text{balance}}$ is the load-balancing auxiliary loss of Switch Transformer [6], and $\mathcal{L}_{\text{pred}}$ is the predictability loss:

$$\mathcal{L}_{\text{pred}} = \frac{1}{|\{(l, h)\}|} \sum_{h \in H} \sum_{l=1}^{L-h} \text{CE}(\hat{z}_{l,h}, y_{l+h}^*) \quad (2)$$

with $H=\{1, 2, 4\}$, the outer factor averaging over all (layer, horizon) pairs, $\hat{z}_{l,h}$ the predictor logits from the layer- l hidden state and y_{l+h}^* a soft target placing uniform $1/k$ mass on the realized top- k set at layer $l+h$. Because the realized top- k targets are argmax-selected and non-differentiable, gradients from Equation (2) reach the backbone *only* through hidden representations: the loss shapes representations, not router weights directly. Optional entropy or stickiness terms are added for controls.

3.4 Isolation protocol

A first-class methodological contribution is our isolation protocol, which mirrors deployment: freeze a trained backbone, train a *fresh* predictor on it (2000–3000 steps), and compare against the same fresh-predictor protocol on the baseline backbone. This is exactly what an engine does when it trains its own predictor against a shipped model, and it prevents any co-adaptation between predictor and backbone from inflating the measured gain.

3.5 Metrics

We report hit@ k per horizon (top- k set overlap $|\text{pred} \cap \text{true}|/k$); router entropy; adjacent-layer routing persistence; maximum expert utilization; validation LM loss on a fixed 16-batch held-out set drawn once before training and shared by every run; and zero-shot downstream accuracy on HellaSwag [20], ARC [3], and PIQA [2] by length-normalized continuation logprob. Unless stated, all results use $n=3$ seeds (init + data order) and we report between-seed standard deviations.

4 Experiments

4.1 Setup and main result

We compare a baseline ($\lambda=0$) against joint training ($\lambda=0.3$) at 25M tokens, horizons $\{1, 2, 4\}$. Controls are fresh post-hoc predictors on the frozen baseline backbone; the joint row is the fresh-predictor-on-joint-backbone isolation configuration of Section 3.4.

Under the stronger ranking-MLP control (3 seeds), baseline reaches 0.903 (0.001) / 0.865 (0.002) / 0.795 (0.001) at $h=1/2/4$ versus joint 0.933 (0.002) / 0.907 (0.004) / 0.847 (0.008): the advantage compresses to $+2.9 \pm 0.2$ / $+4.3 \pm 0.3$ / $+5.2 \pm 0.8$ points (10–25 $\times$ seed noise) but survives intact under an identical predictor. We therefore report +6–7 points (linear) and +3–5 points (ranking) as dual honest effect sizes. The paired quality cost is $+0.023 \pm 0.009$ nats, or $+0.033 \pm 0.016$ with data-order variance included.

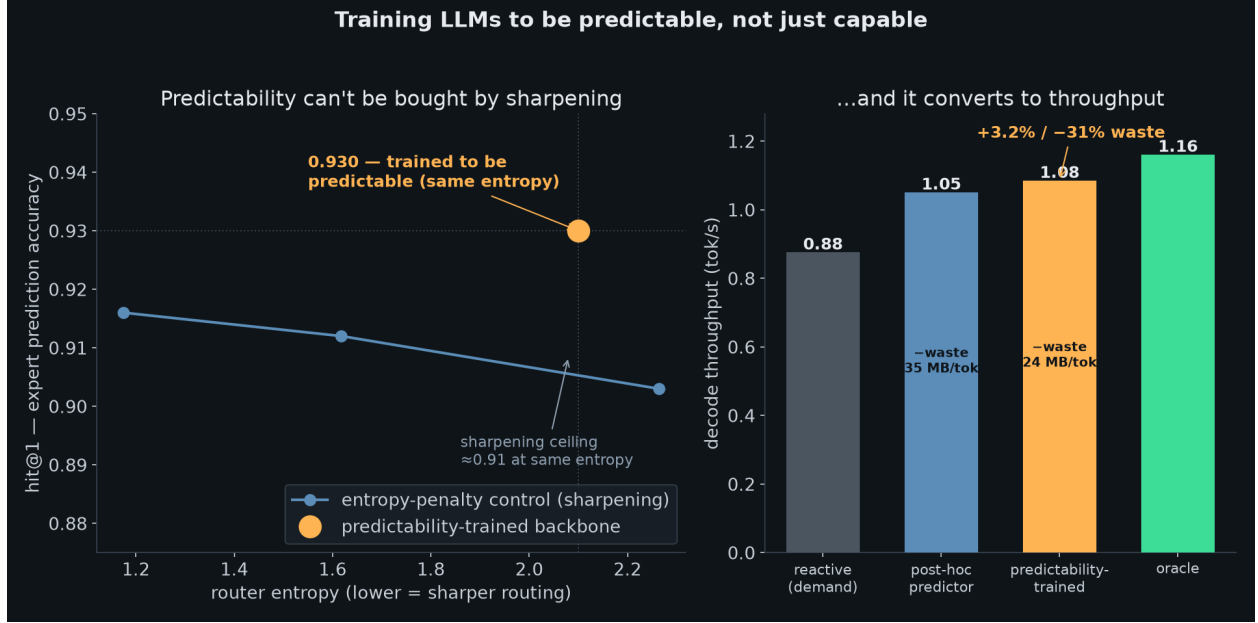


Figure 1: **Structure, not sharpening; predictability converts to throughput.** (left) The entropy-penalty frontier plateaus at ~ 0.91 hit@1 while the predictability-trained backbone reaches 0.930 at matched entropy; (right) decode throughput and misprefetch waste at Colibri-like geometry (corrected simulator).

Table 2: **Robustness.** Joint-vs-control hit@ k advantage (points) and paired quality cost across variance sources and token counts. The advantage is stable to seed noise and grows at $4\times$ tokens, where the quality cost disappears.

Configuration	Δ hit@ k $h=1$	Δ hit@ k $h=4$	Quality cost (nats)
25M tokens, init-only variance	$+6.2 \pm 0.4$	$+6.4 \pm 0.6$	$+0.023 \pm 0.009$
25M tokens, init+data variance	$+6.2 \pm 0.3$	$+6.8 \pm 0.3$	$+0.033 \pm 0.016$
100M tokens ($4\times$)	+6.1	+7.9	−0.019 (free)

4.2 Isolation test: predictability lives in the weights

A fresh post-hoc predictor trained on the frozen joint backbone recovers the co-trained predictor’s accuracy *exactly*: 0.890/0.866/0.796 (fresh) versus 0.887/0.863/0.793 (co-trained) at $h=1/2/4$. Predictability is a property of the weights, not of predictor–backbone co-adaptation. This is the deployable artifact: an engine can ship the jointly-trained model and train its own predictor later, recovering the full gain.

4.3 Structure, not sharpening

One way to make routing easier to predict is to make it more peaked. To test whether our gain is merely sharpening, we train an entropy-penalty ladder $\lambda_{\text{ent}} \in \{0.005, 0.01, 0.05\}$ alongside the baseline and joint models, all evaluated with the ranking-MLP probe (Figure 1). In (entropy, hit@1) coordinates: baseline (2.26, 0.903) $< \lambda=0.005$ (1.62, 0.912) $< \lambda=0.01$ (1.17, 0.916), while the joint model sits at (2.10, 0.930); a near-collapse penalty (entropy 0.28) reaches only 0.883 under the linear probe. Interpolating the penalty frontier to the joint model’s own entropy, sharpening alone yields ~ 0.906 – 0.910 : roughly +2.0–2.4 points of the joint advantage is pure structure. The joint model’s routing is simultaneously informative (high entropy) and forecastable—not merely peaked.

Table 3: **StickyMoE control.** Hit@ k of a post-hoc linear probe on backbones trained with our reimplementaion of the StickyMoE routing-consistency loss. The consistency loss monotonically *reduces* lookahead predictability while raising router entropy.

Backbone	$h=1$ ($\uparrow$)	$h=2$ ($\uparrow$)	$h=4$ ($\uparrow$)	Entropy	Val LM ($\downarrow$)
Baseline	0.822	0.791	0.724	2.264	5.949
Sticky 0.3	0.808	0.774	0.707	2.485	5.945
Sticky 1.0	0.793	0.754	0.682	2.599	5.957
Sticky 3.0	0.774	0.734	0.658	2.676	5.976

4.4 Robustness and growth with training

Table 2 shows the advantage is stable across init and data-order variance and *grows* at $4\times$ training ($h=4$: +7.9), where the quality cost disappears. Baseline routers sharpen naturally with training (entropy $2.26 \rightarrow 1.98$), yet the joint model stays ahead. Zero-shot downstream accuracy (HellaSwag, ARC-Easy, ARC-Challenge, PIQA; $n=1000$ each) is at chance for both arms at 25M tokens—the model is too undertrained for task sensitivity; paired deltas are -0.6 ± 1.0 pp across 12 seed $\times$ benchmark pairs. At 100M tokens the joint model is at or above baseline on all four benchmarks.

4.5 Degeneracy checks

At $\lambda=0.3$ we find no degenerate behavior: router entropy drops only 9% (no collapse), adjacent-layer routing persistence stays at chance (no temporal copying), expert utilization is unchanged, a token-ID $\rightarrow$ expert lookup probe performs equally on both backbones (no memorization), and marginal expert usage is near-uniform (a popularity-trivial predictor hits only 0.15). At $\lambda=1.0$ the degenerate mode appears measurably—entropy -28% and a +0.62 nats quality cost—so the frontier turns, and the turn is visible in our diagnostics.

4.6 StickyMoE control

Table 3 shows that the consistency loss (our reimplementaion of StickyMoE’s objective) monotonically reduces lookahead hit@ k while raising router entropy, and does not stack with our loss (0.864 vs. 0.884 joint-only at $h=1$). Temporal stickiness and lookahead forecastability are different—and here opposed—properties. Scope: we tested only lookahead predictability; StickyMoE’s actual target, temporal-reuse cache locality, is orthogonal and untested here.

5 Systems Analysis

Simulator. We use a trace-driven engine simulator with an honest single-disk FIFO queue: demand fetches have priority, prefetch is issued only into idle compute windows that fit the fetch, and bandwidth conservation is exact. We configure a synthetic Colibri geometry: 75 layers $\times$ 256 experts, top-8 routing, 19 MB int4 experts, 4 ms/layer dense compute, a $C=2000$ -expert cache, Zipf+locality routing, and predictor accuracies from Section 4.

Throughput. With prefetch at $h=1$, decode throughput runs demand $0.876 \rightarrow$ base-control $1.051 \rightarrow$ joint $1.085 \rightarrow$ oracle 1.161 tok/s (linear-control accuracies): +3.2% joint over base, -31% misprefetch waste (24.0 vs. 34.6 MB/tok), capturing $\sim 31\%$ of the base-to-oracle headroom. At ranking-control accuracies (0.903 vs. 0.933, 3 seeds) the conversion is +1.6% and -27% waste—the honest effect size paired with its honest systems value. TTFT (cold cache) improves -11% from prediction generally but only -1 – 3% joint vs. base: it is not the differentiator.

Gating condition. Across a 3×3 latency/bandwidth grid, the joint-vs-base uplift is stable (+2.9–3.2%) whenever prefetch fires. Below ~ 5 GB/s (for 19 MB experts) fetch time exceeds the compute window and *no* prefetch is possible: bandwidth, not latency, gates the regime.

Table 4: **Boundary result.** Hit@ k of post-hoc probes on OLMoE-1B-7B backbones after LoRA fine-tuning with predictability pressure. At $\lambda=0.1$ the isolation test is a tie; at $10\times$ pressure a small effect appears, $2.5\text{--}5\times$ weaker than the pretraining effect.

Post-hoc probe	Baseline backbone	Joint ($\lambda=0.1$)	Joint ($\lambda=1.0$)
Linear $h=1/2/4$	0.799 / 0.781 / 0.752	0.800 / 0.783 / 0.755	0.812 / 0.799 / 0.778
Ranking-MLP (177M) $h=1/2/4$	0.845 / 0.801 / 0.739	0.845 / 0.801 / 0.741	0.847 / 0.807 / 0.753

Real engine. We built a purpose-built O_DIRECT disk-resident engine for the toy model (192 experts $\times$ 2.88 MB, demand-priority disk queue, post-hoc predictor driving a prefetch thread, cache budget $C=64$) and compared joint vs. base backbones in the deployable configuration over 6 decode sequences: +5.0% mean tok/s (range -7% to $+14.5\%$). Per-sequence variance is large at toy scale, so we treat this as qualitative confirmation consistent with the simulator’s direction and magnitude; the quantitative claim stays on the simulator.

6 Boundary: Pretraining-Time Only

Does the method work as a fine-tune on an already-pretrained model? We take OLMoE-1B-7B [13] (64 experts, top-8, 3T pretraining tokens), apply LoRA r16 on attention and experts plus trainable routers, and train 25M tokens at $\lambda \in \{0.1, 1.0\}$ (Table 4). At $\lambda=0.1$ the isolation test is a tie under both probes. At $10\times$ pressure a small backbone effect appears—+1.3/+1.8/+2.6 points under the linear probe and +0.2/+0.6/+1.4 under the ranking probe, growing with horizon—at +0.011 nats quality cost and flat router entropy (3.719 vs. 3.748, so not sharpening). But this is $2.5\text{--}5\times$ weaker than the pretraining effect under the linear probe (and more under the ranking probe, whose ceiling leaves less room): the boundary is dose-dependent, not a strict zero, and the pretraining regime is qualitatively different. For completeness, the *co-trained* predictor at this scale reaches 0.842/0.822/0.794 ($\lambda=0.1$) and 0.844/0.826/0.802 ($\lambda=1.0$) at $h=1/2/4$; its edge over a fresh post-hoc probe on the *same* backbone (+4.2 and +3.2 points at $h=1$) is predictor–backbone co-adaptation, which does not survive predictor retraining—the deployable configuration. The narrowing gap at $10\times$ pressure corroborates that the added pressure moved into the backbone, not just the predictor.

We read this as boundary mapping, not failure: representations must be shaped while plastic, a property the entire training-time prior-art family (StickyMoE, Oracle-MoE) shares. Scope of the negative: one seed, one LoRA rank, one token count (two λ values); heavier fine-tuning (unfrozen blocks, full FT) is untested. Baseline note: a post-hoc predictor on stock OLMoE already reaches 0.845 hit@1—post-hoc predictors are strong on real models, and that is the bar any pretraining application must clear.

7 Limitations

Our positive results rest on one toy from-scratch model (341M parameters, 16 experts, FineWeb-edu only, $\leq 100\text{M}$ tokens). Systems numbers come from a simulator plus a toy engine, not a production engine. Quality is measured by validation LM loss plus chance-level zero-shot downstream probes: we find no consistent joint deficit at 25M tokens and joint $\geq$ baseline on all four benchmarks at 100M tokens, but task sensitivity at this scale is limited. Most results use $n=3$ seeds. The ranking control is our reimplementation of the protocol of Zhu et al. [22]; StickyMoE is tested on our metric only; and the Tier C boundary rests on one seed, one LoRA rank, and one token count (two λ values). Finally, at toy $B=8/C=64$ the joint engine row slightly exceeds the oracle (97.9 vs. 97.4 tok/s)—a queue-interaction effect consistent with the waste ordering (the oracle issues more prefetch traffic), noted for completeness.

8 Conclusion

Predictability is trainable: an MoE optimized jointly for output quality and for the accuracy of an independent expert-activation predictor becomes measurably, structurally, and transferably more forecastable—and the gain converts to throughput and waste reductions wherever an engine has disk slack. The effect must be baked in during pretraining. For pretraining labs targeting disk-resident deployment, predictability pressure is a cheap ($\lambda \approx 0.3$) auxiliary objective whose quality cost is small and vanishes with training; for engine authors, our result defines what a shipped model

can make recoverable. The open question is scale: whether the effect persists at production expert counts and pretraining token budgets, and whether heavier fine-tuning (unfrozen blocks, full fine-tuning) can move the boundary. Data, code, and the full audit trail (including three red-team rounds and all negative results) are available at <https://github.com/splch/predictability-aware-training>.

References

- [1] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. *arXiv:2312.11514*, 2023.
- [2] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of AAAI*, 2020. *arXiv:1911.11641*.
- [3] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. In *arXiv:1803.05457*, 2018.
- [4] Damai Dai, Li Dong, Shuming Ma, Bo Zheng, Zhifang Sui, Baobao Chang, and Furu Wei. Stablemoe: Stable routing strategy for mixture of experts. *arXiv:2204.08396*, 2022.
- [5] Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. *arXiv:2502.12224*, 2025.
- [6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv:2101.03961*, 2021.
- [7] Nishant Gavhane, Arush Mehrotra, Rohit Chawla, and Peter Proenca. Moe-beyond: Learning-based expert activation prediction on edge devices. *arXiv:2508.17137*, 2025.
- [8] Rizhen Hu, Yuan Cao, Boao Kong, Mou Sun, and Kun Yuan. Synergistic intra- and cross-layer regularization losses for MoE expert specialization. *arXiv:2602.14159*, 2026.
- [9] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *51st International Symposium on Computer Architecture (ISCA)*, 2024. *arXiv:2308.12066*.
- [10] JustVugg. Colibrî: Run frontier MoE models on hardware you own. <https://github.com/JustVugg/colibrî>, 2026. Pure-C disk-resident MoE inference engine; PILOT router-lookahead thread.
- [11] Ali Kayyam. Sticky routing: Training MoE models for memory-efficient inference. *arXiv:2607.08780*, 2026. Method referred to as StickyMoE.
- [12] Ang Lv, Jin Ma, Yiyuan Ma, and Siyuan Qiao. Coupling experts and routers in mixture-of-experts via an auxiliary loss. *arXiv:2512.23447*, 2025.
- [13] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, et al. OLMoE: Open mixture-of-experts language models. *arXiv:2409.02060*, 2024.
- [14] Marian-Sergiu Nistor. Halfway speculative decoding: Direct acceptance rate optimization with joint drafter-target training. OpenReview, <https://openreview.net/forum?id=NLdAp0Oizb>, 2026.
- [15] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv:2406.17557*, 2024.
- [16] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.

- [17] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv:2401.14361*, 2024. USENIX ATC 2025.
- [18] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv:2406.06282*, 2024.
- [19] Enda Yu, Dezun Dong, Zhaoning Zhang, Zhe Bai, Weiling Yang, Haojie Wang, Dongsheng Li, Yongwei Wu, and Xiangke Liao. Layerscope: Predictive cross-layer scheduling for efficient multi-batch MoE inference on legacy servers. *arXiv:2509.23638*, 2025. ICS 2026; the prediction-driven scheduling system is named PreScope.
- [20] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of ACL*, 2019. *arXiv:1905.07830*.
- [21] Jixian Zhou, Fang Dong, Ruijun Huang, Hengjie Cao, Mengyi Chen, Yifeng Yang, Anrui Chen, Mingzhi Dong, Yujiang Wang, Dongsheng Li, David A. Clifton, Qin Lv, Rui Zhu, Chun Zhang, Fan Yang, Tun Lu, Ning Gu, and Li Shang. Oracle-MoE: Locality-preserving routing in the oracle space for memory-constrained large language model inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.
- [22] Shien Zhu, Samuel Bohl, Robin Oester, and Gustavo Alonso. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv:2511.10676*, 2025.
- [23] Xiongwei Zhu, Xiaojian Liao, Tianyang Jiang, Yusen Zhang, Liang Wang, and Limin Xiao. Remoe: Boosting expert reuse through router fine-tuning in memory-constrained MoE LLM inference. *arXiv:2605.27081*, 2026. ICML 2026.